



Autor: Mateo Quintela Trillo

Tutor: Jesús Ángel Pérez-Roca Fernández



TrainerForge

Resumen del proyecto

TrainerForge es una aplicación web concebida como herramienta de apoyo para jugadores de Pokémon, orientada a facilitar la creación, organización y consulta de equipos durante el transcurso de una partida. El proyecto surge de la necesidad de unificar en una única plataforma distintos recursos que habitualmente se encuentran dispersos en notas, hojas de cálculo, documentos y múltiples páginas web, lo que complica el seguimiento del progreso y la planificación de estrategias. Frente a otros portales centrados en la acumulación masiva de información o en ámbitos muy concretos de la franquicia, TrainerForge se plantea como un acompañante a la partida, diseñado para mostrar únicamente los datos y utilidades realmente relevantes en el momento de juego.

La aplicación incorpora como base un Pokédex con filtros, páginas de detalle de cada Pokémon, un generador de equipos y una sección de contacto, constituyendo así un núcleo funcional orientado a cubrir las necesidades principales del usuario desde las primeras versiones. Entre sus rasgos diferenciales destaca la intención de emplear recursos propios generados de forma procedural, así como una arquitectura pensada para facilitar la incorporación de futuras actualizaciones y nuevas funcionalidades sin comprometer la estructura principal del sistema. Este enfoque permite ampliar progresivamente la plataforma con elementos como perfiles de usuario, almacenamiento de contenido, simulación de combates, logros o funciones sociales, de forma coherente con la evolución prevista del proyecto.

Desde el punto de vista técnico, TrainerForge se plantea como una aplicación web full stack desarrollada con Spring Boot en el backend, React y TypeScript en el frontend, y PostgreSQL como sistema de base de datos. Además, el proyecto contempla el uso de herramientas de apoyo al desarrollo y despliegue como Git, GitHub y Docker, junto con tecnologías de testing y diseño que contribuyen a una construcción ordenada, mantenible y escalable. En conjunto, TrainerForge aspira a ofrecer una solución útil, clara y extensible para la gestión de equipos Pokémon, combinando organización de información, soporte al jugador y capacidad de crecimiento futuro.

Abstract

TrainerForge is a web application designed to support Pokémon players while they build, organize and manage their teams throughout a playthrough. The project was created in response to a practical need: centralizing in a single platform the information, planning tools and team-building resources that are usually scattered across notes, spreadsheets and multiple web pages. Rather than functioning as a general-purpose wiki, TrainerForge is conceived as a companion tool focused on the specific needs of team creation, progress tracking and strategic consultation during gameplay.

The application has been designed with scalability in mind, allowing future updates and new content to be incorporated easily as the project grows. This approach makes it possible to expand the platform over time with additional features, user tools and new Pokémon-related resources without compromising its core structure. From a technical perspective, the project is planned as a full-stack web application built with Spring Boot on the backend, React and TypeScript on the frontend, and PostgreSQL as the database system, aiming to deliver a functional, organized and extensible solution for players.

Palabras clave

- **TrainerForge:** nombre de la aplicación web desarrollada como herramienta de apoyo para jugadores de Pokémon durante una partida.
- **Pokémon:** universo temático sobre el que se construye el proyecto y del que procede toda la lógica funcional de la aplicación.
- **Aplicación web:** tipo de solución desarrollada, accesible desde navegador y pensada para centralizar información y utilidades en una única plataforma.
- **Full stack:** enfoque de desarrollo empleado, al abarcar tanto frontend como backend y base de datos dentro del mismo proyecto.
- **Acompañante a la partida:** concepto central de TrainerForge, definido como una herramienta pensada para estar abierta mientras el usuario juega.
- **Gestión de equipos:** función principal del sistema, orientada a crear, organizar, guardar y consultar equipos Pokémon.
- **Planificación estratégica:** finalidad práctica de la aplicación, al ayudar al usuario a preparar equipos y consultar datos útiles para la partida.
- **Centralización de información:** objetivo del proyecto, que busca reunir en una sola web recursos que normalmente están dispersos en varias fuentes.
- **Pokédex:** módulo principal que ofrece un listado de Pokémon con información organizada para su consulta.
- **Filtros avanzados:** funcionalidad aplicada al Pokédex para buscar Pokémon según estadísticas y distintos criterios de detalle.
- **Ficha de Pokémon:** página de detalle individual con datos como descripción, tipos, estadísticas base, juegos y movimientos.
- **Generador de equipos:** sección destinada a construir equipos de hasta seis Pokémon y configurar sus datos de combate.
- **Movimientos:** uno de los elementos configurables dentro de cada equipo Pokémon creado por el usuario.
- **Habilidades:** atributo de combate contemplado en la construcción detallada de equipos.
- **Objetos equipados:** componente estratégico que puede asociarse a cada Pokémon dentro del generador de equipos.
- **Modalidades de juego:** variantes contempladas en el proyecto, como equipos temáticos, monotype o retos especiales.

- Nuzlocke: ejemplo de reto autoimpuesto para el que el generador de equipos se plantea como herramienta útil.
- Monotype: modalidad de equipo basada en un único tipo, contemplada entre los usos específicos del generador.
- Descarga de equipos: posibilidad de exportar el contenido generado en distintos formatos para su consulta fuera de la web.
- Formulario de contacto: apartado mediante el cual los usuarios pueden enviar sugerencias u opiniones al desarrollador.
- Tarjeta de Entrenador: página de perfil de usuario inspirada en los carnés de la Liga Pokémon presentes en la franquicia.
- Perfil de usuario: sistema previsto para almacenar información personal y asociar contenido generado dentro de la plataforma.
- Guardado de contenido: funcionalidad orientada a conservar equipos, registros y otros elementos vinculados a cada usuario.
- Logros: sistema previsto para recompensar acciones del usuario mediante sellos visibles u ocultos.
- Simulador de combate: módulo planteado para estimar el ganador entre dos equipos en distintas modalidades.
- Simulación representativa: criterio del simulador, cuyo resultado se basa en parámetros orientativos y no en una reproducción completa de la lógica del juego.
- Funciones sociales: conjunto de mejoras futuras orientadas a permitir interacción entre usuarios y compartir contenido.
- GTS: referencia al sistema de intercambios inspirado en Pokémon HOME, previsto como futura funcionalidad social.
- Modo oscuro: mejora de interfaz prevista para adaptar la experiencia visual a las preferencias del usuario y al dispositivo.
- Escalabilidad: característica deseable del proyecto, al estar planteado para facilitar la incorporación de nuevas funciones y contenido.
- Desarrollo modular: enfoque de construcción basado en bloques funcionales que permite ampliar la aplicación por versiones.
- Actualizaciones evolutivas: idea asociada al roadmap del proyecto, que prevé la incorporación progresiva de nuevas capacidades.

- Roadmap: planificación de versiones y objetivos funcionales prevista para la evolución de TrainerForge.
- Recursos procedurales: enfoque planteado para generar elementos multimedia propios y reducir dependencia de materiales externos.
- Frontend: capa cliente de la aplicación, responsable de la interfaz y de la interacción con el usuario.
- Backend: capa servidor encargada de la lógica de negocio, seguridad y acceso a datos.
- Base de datos: componente encargado del almacenamiento estructurado de la información del sistema.
- Java: lenguaje principal elegido para el desarrollo del backend.
- Spring Boot: framework utilizado para construir la API REST del proyecto.
- Spring Data JPA: tecnología prevista para la persistencia y el acceso a datos en el backend.
- Spring Security: framework utilizado para gestionar la seguridad de la aplicación.
- JWT: mecanismo de autenticación planteado para el control de acceso de usuarios.
- Lombok: biblioteca elegida para reducir código repetitivo en la parte backend.
- Maven: gestor de dependencias utilizado en el entorno Java del proyecto.
- TypeScript: lenguaje principal seleccionado para el desarrollo del frontend.
- React: biblioteca o framework de interfaz utilizado para construir la capa visual de la aplicación.
- Vite: herramienta elegida para la construcción y el arranque del proyecto frontend.
- Tailwind CSS: framework de estilos utilizado para definir la interfaz visual.
- PostgreSQL: sistema de gestión de bases de datos elegido para almacenar la información del proyecto.
- Modelo Entidad-Relación: representación estructural de la base de datos requerida y contemplada en la documentación técnica.

- API REST: modelo de comunicación previsto entre frontend y backend.
- Autenticación: proceso de identificación de usuarios contemplado mediante Spring Security y JWT.
- Control de versiones: práctica de desarrollo aplicada al proyecto mediante herramientas específicas.
- Git: sistema de control de versiones utilizado durante el desarrollo.
- GitHub: repositorio remoto previsto para almacenar, documentar y hacer seguimiento del proyecto.
- Docker: tecnología planteada para facilitar despliegue y contenedorización.
- Vercel: plataforma prevista para el despliegue web del proyecto.
- Testing: conjunto de pruebas necesarias para validar el funcionamiento del sistema.
- JUnit 5: herramienta propuesta para las pruebas del backend.
- Jest: herramienta propuesta para las pruebas del frontend.
- Thunder Client: utilidad contemplada para probar endpoints REST durante el desarrollo.
- Figma: herramienta empleada para la elaboración de wireframes, mockups y otros recursos de diseño.
- draw.io: herramienta destinada a la elaboración de diagramas del proyecto.
- Wireframes: representaciones preliminares de la interfaz exigidas en la memoria y ya contempladas en el anteproyecto.
- Mockups: diseños visuales más detallados que forman parte de la documentación de diseño requerida.
- Guía de estilos: documento visual necesario para justificar decisiones de diseño, coherencia estética y usabilidad.
- Usabilidad: criterio de diseño relevante en una aplicación pensada para consulta rápida durante la partida.
- Accesibilidad: aspecto que la memoria debe contemplar dentro del diseño web y sus pruebas asociadas.
- UX/UI: dimensión visual y de experiencia de usuario que estructura la presentación y el uso general de la plataforma.

- Diseño responsive: principio implícito en una aplicación web moderna orientada a distintos dispositivos y contextos de uso.
- Semántica HTML: requisito técnico general aplicable al desarrollo web del proyecto.
- Administración de contenidos: exigencia habitual en el TFC para poder modificar información desde la propia aplicación.
- Documentación técnica: parte de la memoria donde se recogen tecnologías, decisiones de diseño y estructura del sistema.
- Memoria de proyecto: documento académico obligatorio que debe incluir resumen, abstract, palabras clave y el resto de apartados exigidos.

Motivación y Objetivos

TrainerForge nace de una necesidad concreta detectada durante el uso continuado de videojuegos de la franquicia Pokémon y, más concretamente, durante el proceso de creación, organización y seguimiento de equipos a lo largo de una partida. A medida que aumenta la experiencia del jugador y se amplía el número de partidas realizadas, también crece la cantidad de información que resulta útil conservar: posibles integrantes de un equipo, combinaciones temáticas, movimientos, habilidades, objetos, estrategias, juegos de procedencia o simples anotaciones sobre decisiones tomadas en cada partida. Cuando toda esa información se gestiona de forma dispersa, mediante notas sueltas, hojas de cálculo, documentos de texto y múltiples páginas web abiertas al mismo tiempo, el proceso se vuelve progresivamente más lento, menos claro y bastante más incómodo de mantener.

Precisamente en ese contexto surge la idea de desarrollar TrainerForge. El proyecto parte de una experiencia real de uso y no de una necesidad puramente teórica, ya que su origen se encuentra en una problemática que aparece de forma recurrente al intentar documentar y planificar equipos Pokémon con cierto nivel de detalle. Con el paso del tiempo, las soluciones improvisadas que podían resultar suficientes en un primer momento dejan de ser adecuadas, porque no ofrecen una estructura homogénea, no facilitan la consulta rápida de la información y tampoco permiten mantener una organización sólida cuando el volumen de datos empieza a crecer.

Desde esta perspectiva, TrainerForge se concibe como una respuesta específica a un problema cotidiano dentro de un ámbito muy concreto: el seguimiento y la gestión de equipos Pokémon durante la partida. No se trata, por tanto, de desarrollar una web genérica sobre la franquicia, sino de construir una herramienta con un propósito definido, centrada en acompañar al usuario mientras juega y en proporcionarle únicamente aquellos recursos que realmente necesita en ese momento. Esta orientación práctica constituye uno de los pilares fundamentales del proyecto, ya que condiciona tanto la selección de funcionalidades como la forma en la que la información debe organizarse y mostrarse.

Para comprender mejor el enfoque adoptado, conviene señalar también qué no pretende ser TrainerForge. El anteproyecto establece con claridad que no se ha concebido como una wiki de Pokémon, puesto que ese tipo de portales se caracteriza por ofrecer grandes volúmenes de información enciclopédica, enlazar contenidos de muy diversa naturaleza y abarcar prácticamente cualquier aspecto relacionado con la franquicia. Aunque TrainerForge también maneja información relevante, su valor no reside en la acumulación masiva de datos, sino en la selección, estructuración y presentación de contenidos útiles para crear equipos, consultar detalles concretos y registrar el progreso de una partida de forma ordenada.

Del mismo modo, tampoco se plantea como una página dedicada a una única vertiente del universo Pokémon. Existen portales especializados en estrategia competitiva, otros centrados en anime, otros orientados al coleccionismo y otros enfocados a bases de datos puramente informativas, pero TrainerForge adopta una posición intermedia y más funcional, al reunir distintas utilidades relacionadas con la experiencia práctica de jugar. Esa decisión responde a una motivación clara: ofrecer una herramienta transversal que no obligue al usuario a saltar constantemente entre plataformas distintas para completar tareas que, en realidad, forman parte de una misma actividad.

En consecuencia, la mejor forma de definir TrainerForge es como un acompañante a la partida. La aplicación está pensada para permanecer abierta durante el desarrollo de una sesión de juego, permitiendo construir equipos, consultar fichas de Pokémon, revisar datos importantes y reflejar determinados progresos conforme avanza la partida. Esta idea de acompañamiento permanente aporta personalidad al proyecto y lo diferencia de otras soluciones más estáticas, ya que pone el foco en el contexto de uso, en la inmediatez de la consulta y en la utilidad real de cada apartado dentro de la experiencia del jugador.

La motivación del proyecto no se limita, por tanto, al interés por la franquicia Pokémon, sino que se apoya también en una inquietud relacionada con el diseño de herramientas digitales bien enfocadas a un problema concreto. Desde el punto de vista del desarrollo de aplicaciones web, resulta especialmente interesante abordar un proyecto que exige transformar una necesidad personal y poco estructurada en un sistema organizado, escalable y utilizable por terceros. Esto convierte a TrainerForge en una propuesta adecuada para un Trabajo de Fin de Ciclo, ya que combina una base temática reconocible con retos técnicos, funcionales y de diseño propios de una aplicación web completa.

Otro de los motivos que justifican la elección de este proyecto es su capacidad para integrar de forma coherente distintas áreas del desarrollo web. El anteproyecto define una arquitectura full stack con tecnologías diferenciadas para frontend, backend, seguridad, persistencia de datos, control de versiones, despliegue y testing, lo que permite trabajar con un alcance suficientemente amplio y alineado con la formación del ciclo de Desarrollo de Aplicaciones Web. En este sentido, TrainerForge no solo responde a una motivación temática, sino también a la voluntad de aplicar de forma práctica conocimientos sobre APIs, interfaces de usuario, autenticación, modelado de bases de datos, pruebas y publicación de una aplicación funcional.

La elección del proyecto también se ve reforzada por su potencial de crecimiento. Desde sus fases iniciales, TrainerForge se ha planteado como una plataforma preparada para admitir nuevas actualizaciones y ampliaciones de contenido con relativa facilidad, de manera que la base principal pueda evolucionar sin necesidad de rehacer por completo la estructura de la aplicación. Esta forma de entender el desarrollo resulta especialmente valiosa en un proyecto de estas características, ya que la franquicia sobre la que se apoya es muy amplia y permite incorporar, de forma progresiva, nuevas funciones, nuevas secciones y nuevos datos sin agotar rápidamente el recorrido de la herramienta.

Esa previsión de crecimiento aparece reflejada en el propio roadmap del anteproyecto. En una primera etapa se plantean funcionalidades esenciales como el Pokédex con filtros, las páginas de detalle de cada Pokémon, el generador de equipos y la sección de contacto; posteriormente se incorporan elementos como la Tarjeta de Entrenador, el guardado de contenido y un simulador de combate; y, en fases posteriores, se contemplan sistemas de logros, funciones sociales, simulación de intercambios y mejoras generales de experiencia de usuario. Esta evolución planificada demuestra que el proyecto no se ha concebido como un desarrollo cerrado y rígido, sino como una base sólida sobre la que seguir construyendo valor funcional.

Junto a ello, el anteproyecto incorpora una decisión especialmente relevante desde el punto de vista conceptual: el uso de recursos propios basados en los oficiales. Esta medida se justifica por el hecho de que The Pokémon Company no respalda oficialmente este tipo de desarrollos realizados por fans, de modo que el proyecto opta por reducir dependencias mediante la generación procedural de determinados recursos multimedia. Más allá de su dimensión práctica, esta decisión aporta identidad al proyecto y lo dota de un componente técnico adicional, al obligar a pensar soluciones propias allí donde otras páginas podrían limitarse a reutilizar materiales ya existentes.

La motivación principal, en cualquier caso, sigue siendo la de mejorar una experiencia de uso que actualmente resulta fragmentada. Crear equipos Pokémon con cierto nivel de profundidad implica consultar numerosos datos, comparar opciones, recordar decisiones previas y mantener una visión clara del conjunto, y ese proceso pierde fluidez cuando depende de herramientas inconexas entre sí. TrainerForge pretende corregir esa fragmentación mediante una plataforma unificada que reúna consulta, organización y construcción de equipos dentro de un mismo entorno web.

Desde un punto de vista más amplio, el proyecto responde también a una idea de calidad en el desarrollo de software: una aplicación útil no se define únicamente por la cantidad de funciones que incorpora, sino por la coherencia con la que esas funciones resuelven una necesidad real.

Por ello, el diseño de TrainerForge no persigue abarcar todo lo relacionado con Pokémon, sino centrar sus esfuerzos en aquello que verdaderamente aporta valor al usuario durante la partida. Esta decisión de acotar el alcance resulta importante tanto a nivel funcional como académico, porque permite justificar mejor cada apartado de la aplicación y mantener una línea clara entre problema identificado y solución propuesta.

Además, el proyecto ofrece un marco adecuado para trabajar aspectos que la guía del TFC considera esenciales en una aplicación web, como el diseño, la usabilidad, la estructura de datos, la documentación técnica y la correcta justificación de las decisiones adoptadas. Al no tratarse de una idea abstracta, sino de una herramienta con casos de uso muy concretos, resulta más sencillo fundamentar por qué se necesitan determinadas vistas, por qué ciertas relaciones de base de datos son relevantes y por qué unas funciones tienen prioridad sobre otras. De este modo, la motivación del proyecto no solo se apoya en el interés temático, sino también en su capacidad para convertirse en una propuesta técnicamente consistente y académicamente defendible.

En lo relativo a los objetivos que se persiguen con su desarrollo, el primero de ellos consiste en crear una aplicación web capaz de centralizar la información y las herramientas necesarias para la planificación y gestión de equipos Pokémon. A partir de ese objetivo general, se derivan otros más concretos, como ofrecer un sistema de consulta eficaz mediante un Pokédex filtrable, permitir la creación detallada de equipos, facilitar el almacenamiento estructurado de información asociada al usuario y sentar una base técnica que permita futuras ampliaciones sin comprometer la estabilidad del sistema. También se persigue que la aplicación presente una interfaz clara y adecuada al contexto de uso, de forma que el acceso a la información resulte rápido, ordenado y comprensible para el usuario.

En definitiva, TrainerForge se elige como proyecto porque une una necesidad real, una temática sólida y un recorrido técnico amplio dentro del ámbito del desarrollo web. Su motivación nace de una experiencia concreta de uso, pero su planteamiento trasciende lo personal para transformarse en una herramienta estructurada, escalable y orientada a resolver un problema identificable para otros jugadores de la franquicia. Por ello, el desarrollo de esta aplicación no solo busca materializar una idea funcional, sino también demostrar la capacidad de analizar una necesidad, traducirla a requisitos de software y convertirla en una solución web coherente, mantenible y con proyección de crecimiento.

Estado del arte

El desarrollo de TrainerForge se sitúa dentro de un ecosistema ya consolidado de páginas y herramientas web relacionadas con la franquicia Pokémon. En Internet existen numerosas soluciones orientadas a la consulta de datos, a la simulación de combates o a la planificación de equipos, lo que obliga a delimitar con claridad el espacio que pretende ocupar este proyecto y a justificar qué aporta frente a opciones ya disponibles. Por ello, el análisis del estado del arte debe centrarse en aquellas aplicaciones que comparten, al menos parcialmente, el mismo ámbito funcional: bases de datos de Pokémon, constructores de equipos y plataformas de simulación o planificación.

En términos generales, las soluciones existentes pueden agruparse en tres grandes bloques. En primer lugar, se encuentran las páginas de consulta de información, cuyo objetivo principal es ofrecer datos extensos sobre especies, movimientos, habilidades, objetos, generaciones y mecánicas del juego. En segundo lugar, aparecen las herramientas orientadas a la construcción y análisis de equipos, centradas en coberturas, sinergias, debilidades o planificación previa de partidas. Finalmente, existe un tercer grupo formado por simuladores y plataformas competitivas, cuyo propósito principal es reproducir combates y probar equipos en escenarios de juego más complejos.

Dentro del primer grupo, una de las referencias más evidentes es Pokémon Database. Su Pokédex ofrece páginas detalladas para cada Pokémon con datos de estadísticas, descripciones, movimientos, evoluciones y otros elementos relevantes, lo que la convierte en una fuente muy completa para la consulta individual de información. Además, el repositorio de datos asociado indica que el sitio se apoya en una biblioteca de datos propia utilizada para mantener actualizada la información del portal, lo que sugiere una gestión estructurada del contenido y una clara separación entre datos y capa de presentación.

El principal punto fuerte de Pokémon Database reside en la amplitud y profundidad de su contenido. Para un usuario que necesita consultar información exhaustiva sobre una criatura concreta, sus movimientos o sus evoluciones, se trata de una herramienta muy eficaz, rápida y suficientemente consolidada. Sin embargo, precisamente esa orientación enciclopédica constituye también una de sus limitaciones en comparación con TrainerForge, ya que no está concebida como un acompañante a la partida ni como un espacio centralizado para construir, organizar y conservar equipos personalizados con una lógica de uso continuado durante el progreso del jugador.

Desde la perspectiva del proyecto, Pokémon Database representa una alternativa parcial, pero no una solución equivalente. Aporta una referencia útil sobre cómo estructurar un Pokédex y cómo presentar grandes cantidades de información de forma accesible, pero no cubre el núcleo funcional que TrainerForge pretende desarrollar, centrado en la creación de equipos, el registro del progreso y la futura asociación de datos al perfil del usuario. Por ello, puede considerarse una inspiración válida en la dimensión informativa del proyecto, aunque no en su propuesta de valor principal.

En el ámbito de las herramientas específicas de planificación de equipos, una de las más conocidas es Marriland Team Builder. Esta herramienta permite introducir equipos de hasta seis Pokémon y comprobar de un vistazo las debilidades y resistencias del conjunto, lo que la ha convertido en una referencia habitual para usuarios que desean detectar carencias defensivas de forma rápida. Su enfoque es claro, directo y muy visual, lo que favorece un uso ágil y centrado en un único problema: evaluar la cobertura del equipo frente a distintos tipos.

La principal fortaleza de Marriland Team Builder es precisamente su simplicidad. Al reducir su alcance a la lectura visual de fortalezas y debilidades, consigue ofrecer una experiencia muy inmediata y fácil de interpretar, especialmente útil en etapas tempranas de planificación de un equipo. No obstante, esa misma especialización limita su utilidad en contextos más amplios, ya que no está planteada como plataforma de gestión integral, no articula un sistema de perfil o de progresión asociado al usuario y tampoco persigue centralizar otros recursos relevantes para una partida, como el registro de equipos guardados, el seguimiento de capturas o la consulta contextualizada de contenido.

Frente a ello, TrainerForge plantea una visión más amplia. Mientras que Marriland resuelve correctamente una necesidad concreta, TrainerForge pretende integrar esa lógica de apoyo estratégico dentro de una aplicación más completa, capaz de combinar consulta de datos, construcción detallada de equipos, almacenamiento de contenido y futuras ampliaciones funcionales dentro de un mismo entorno. En consecuencia, Marriland puede entenderse como un antecedente importante para el módulo de análisis de equipos, pero no como una solución que cubra la totalidad del enfoque previsto en este proyecto.

Otra referencia relevante dentro del ámbito de planificación es Pokémon Team Planner, del que existen repositorios públicos tanto para frontend como para backend. La descripción del frontend indica que se trata de una aplicación en React para planificar equipos de hasta seis Pokémon para un juego seleccionado, con filtros por tipo y exclusividad de versión, además de opciones para guardar equipos. Por su parte, el backend se describe como una API en Node.js orientada a servir datos de Pokémon, manejar equipos y gestionar autenticación de usuarios, apoyándose en una base de datos MongoDB.

En este caso, la comparación con TrainerForge resulta especialmente útil, porque se trata de una solución más próxima a la idea de planificar equipos dentro del contexto de los juegos principales. Entre sus puntos fuertes se encuentran la incorporación de autenticación, la posibilidad de guardar o compartir equipos y la existencia de una arquitectura separada entre cliente y servidor, lo que demuestra una aproximación full stack real. Sin embargo, la información disponible sugiere un enfoque más funcional que conceptual, centrado en planificar y compartir equipos, mientras que TrainerForge persigue además construir una identidad propia como acompañante a la partida, articulando módulos distintos pero conectados entre sí, como Pokédex, perfiles tipo Tarjeta de Entrenador, simulación, progreso y funciones sociales futuras.

También en el terreno de la preparación de equipos debe mencionarse Pikalytics, una plataforma orientada principalmente al juego competitivo y al análisis basado en datos de uso real. Sus páginas ofrecen información sobre builds, movimientos, objetos, habilidades, compañeros frecuentes y distribuciones de estadísticas derivadas de metajuegos y torneos, lo que la convierte en una herramienta muy valiosa para jugadores competitivos. Además, publicaciones relacionadas con su team builder explican que utiliza algoritmos que tienen en cuenta relevancia en el metajuego y frecuencia de aparición conjunta entre compañeros, reforzando así su orientación analítica y estadística.

El punto fuerte de Pikalytics es su sofisticación dentro del ámbito competitivo. Allí donde otras herramientas ofrecen análisis básicos de tipos o de composición, Pikalytics aporta información derivada del comportamiento real del metajuego, lo que permite construir equipos alineados con tendencias actuales y con datos de uso contrastados. Sin embargo, esa especialización competitiva también la aleja del propósito principal de TrainerForge, que no busca centrarse exclusivamente en formatos competitivos, sino en el acompañamiento general al jugador durante partidas normales, retos autoimpuestos o procesos personales de organización de equipos.

Por tanto, Pikalytics debe entenderse más como una referencia avanzada para futuras mejoras analíticas que como un competidor directo del núcleo actual de TrainerForge. Sus métodos resultan especialmente interesantes si en el futuro se desea enriquecer la aplicación con sugerencias automáticas, evaluación de sinergias o recomendaciones más inteligentes, pero su propuesta de valor actual responde a un perfil de usuario distinto y a un uso más cercano al juego competitivo estructurado.

El caso de Pokémon Showdown es diferente, ya que se trata de una de las plataformas más consolidadas del ecosistema Pokémon en lo relativo a simulación de combates. Su repositorio oficial describe el proyecto como sitio web para combatir, biblioteca JavaScript para simular combates y obtener datos de Pokédex, API web y servidor para alojar comunidades o modos de juego propios. Además, el propio repositorio señala que simula combates individuales, dobles y triples a lo largo de todas las generaciones disponibles hasta la novena, lo que evidencia un nivel de complejidad y madurez muy elevado.

La gran fortaleza de Pokémon Showdown es su profundidad técnica. No solo permite construir equipos y probarlos, sino que implementa una simulación de combate extremadamente completa, con una arquitectura de cliente, servidor, biblioteca y protocolos de comunicación bien definidos, y con una base de código dominada por TypeScript. Frente a esto, TrainerForge no pretende competir directamente en fidelidad de simulación ni en amplitud comunitaria, ya que su simulador previsto es representativo y deliberadamente más sencillo, al basarse en datos fijos sin reproducir toda la lógica avanzada de un combate real.

Esa diferencia es importante, porque delimita con claridad el alcance del proyecto. TrainerForge no busca reemplazar a una plataforma competitiva madura, sino ofrecer una experiencia distinta, más orientada a la organización, la consulta contextual y el seguimiento personal de equipos. Aun así, Pokémon Showdown constituye una referencia técnica de gran valor para comprender cómo estructurar herramientas relacionadas con equipos, validación de datos y simulación, especialmente si en el futuro se desea ampliar la complejidad del módulo de combate o estudiar formas de interoperabilidad conceptual.

En cuanto a las tecnologías observadas en este estado del arte, pueden apreciarse varias tendencias comunes. Por un lado, las herramientas modernas de planificación tienden a utilizar arquitecturas web separadas entre frontend y backend, con bibliotecas de interfaz como React en el cliente y entornos JavaScript o Node.js en el servidor.

Por otro, proyectos más complejos y maduros como Pokémon Showdown muestran un uso intensivo de TypeScript y una organización modular capaz de soportar lógica de negocio avanzada, mientras que otras iniciativas de consulta de datos recurren a estrategias basadas en bibliotecas de datos propias, ficheros CSV, JSON o carga optimizada en memoria.

TrainerForge adopta una solución tecnológica diferente, basada en Spring Boot para el backend, React y TypeScript para el frontend y PostgreSQL como sistema de base de datos. Esta elección presenta varias ventajas respecto a las alternativas observadas. El uso de Spring Boot permite construir una API REST robusta, con una capa de seguridad bien integrada mediante Spring Security y JWT, mientras que PostgreSQL ofrece un modelo relacional sólido y adecuado para gestionar entidades conectadas entre sí, algo especialmente relevante en una aplicación que debe manejar Pokémon, equipos, usuarios, juegos, movimientos, objetos y futuras relaciones sociales.

Como alternativas tecnológicas, podrían haberse considerado opciones como un stack completo en JavaScript con Node.js, Express y MongoDB, similar al empleado por algunas herramientas de planificación existentes. Esta vía habría facilitado la unificación del lenguaje en cliente y servidor y podría resultar adecuada para desarrollos rápidos o para modelos de datos menos relacionales. Sin embargo, para un proyecto como TrainerForge, que requiere consistencia estructural, modelado claro de relaciones y una base mantenible de cara a futuras ampliaciones, la combinación de Spring Boot y PostgreSQL ofrece una base especialmente apropiada.

También sería posible plantear alternativas más ligeras apoyadas directamente en APIs públicas como PokeAPI o en ficheros estáticos procesados en el frontend, como muestran algunos proyectos de consulta de datos. Estas opciones simplifican ciertas fases iniciales de desarrollo y reducen el esfuerzo de mantenimiento de datos propios. No obstante, presentan inconvenientes importantes para una herramienta como TrainerForge, ya que limitan el control sobre la estructura de la información, dificultan la personalización avanzada del modelo de datos y pueden generar dependencias externas poco deseables en un proyecto que pretende evolucionar con módulos propios y con una identidad funcional diferenciada.

A la vista de este análisis, puede afirmarse que no existe una herramienta única que reúna exactamente el mismo enfoque que TrainerForge. Algunas soluciones destacan por su volumen de información, otras por su análisis estratégico y otras por la potencia de su simulación competitiva, pero ninguna combina de forma equilibrada la idea de acompañamiento a la partida, la construcción y conservación de equipos, la consulta contextualizada de datos y la preparación para futuras ampliaciones de contenido dentro de un único sistema. En este sentido, el estado del arte confirma que TrainerForge no surge para duplicar una herramienta existente, sino para ocupar un espacio intermedio poco cubierto por las soluciones actualmente disponibles

Resumen del proyecto

El problema que aborda TrainerForge no consiste en la ausencia de información sobre Pokémon en Internet, sino en la falta de una herramienta que reúna de forma coherente las funciones necesarias para planificar, construir, consultar y conservar equipos durante el transcurso de una partida. El análisis del estado del arte ha mostrado que existen soluciones muy competentes en ámbitos concretos, como las bases de datos enciclopédicas, los constructores de equipos centrados en coberturas o los simuladores competitivos, pero también ha puesto de manifiesto que esas herramientas suelen responder a necesidades parciales y obligan al usuario a cambiar continuamente de contexto. En consecuencia, el caso de estudio de TrainerForge se centra en diseñar una aplicación que elimine esa fragmentación y transforme un conjunto disperso de recursos en una experiencia unificada, estable y orientada al uso real durante la partida.

La problemática inicial puede resumirse en varios puntos. En primer lugar, la información relevante para crear un equipo suele encontrarse repartida entre distintas páginas, documentos y notas, lo que obliga al usuario a consultar varias fuentes para completar tareas que deberían resolverse en un solo entorno. En segundo lugar, muchas herramientas existentes están pensadas para análisis puntuales, pero no para acompañar un proceso continuado de juego, de modo que no facilitan el seguimiento del progreso, la evolución de un equipo ni la conservación ordenada de decisiones tomadas a lo largo del tiempo. En tercer lugar, cuando una aplicación sí cubre parte de esa necesidad, acostumbra a hacerlo desde una perspectiva muy concreta, sin integrar adecuadamente consulta, organización, personalización y persistencia de datos.

A partir de esa base, TrainerForge se plantea como una respuesta estructurada en varios bloques funcionales que atacan directamente cada una de esas carencias. La aplicación no se limita a ofrecer una única función principal, sino que articula un conjunto de módulos conectados entre sí, de manera que el usuario pueda pasar de la consulta de información a la construcción de equipos, y de ahí al guardado, a la simulación o a la personalización del perfil, sin abandonar la misma plataforma. Esa continuidad entre acciones constituye una de las mejoras más claras frente a muchas de las soluciones revisadas, ya que reduce fricción, favorece la organización y ofrece una experiencia mucho más coherente.

Uno de los pilares del proyecto es el Pokédex, entendido no como una base de datos enciclopédica generalista, sino como un módulo de consulta enfocado a la utilidad práctica. Su función no es competir en volumen de contenido con grandes portales especializados, sino presentar de forma clara y accesible la información necesaria para tomar decisiones durante la construcción de equipos y el avance en una partida.

Para ello, se plantea una lista clásica de Pokémon acompañada de filtros por diferentes estadísticas y detalles, lo que permite acotar búsquedas con rapidez y adaptar la consulta al contexto concreto del usuario. Esta decisión mejora el enfoque de las páginas puramente informativas, porque desplaza el peso desde la acumulación masiva de datos hacia la selección eficiente de información relevante.

La mejora se hace todavía más visible en la página de detalle de cada Pokémon. En lugar de presentar únicamente una ficha descriptiva, TrainerForge plantea una vista compuesta por imagen del Pokémon, tabla de datos técnicos, descripción, listado de juegos en los que aparece, movimientos que puede aprender, estadísticas base y objetos relacionados. De esta manera, el usuario puede resolver en una sola pantalla consultas que normalmente requerirían visitar varias secciones o incluso varias webs distintas. La aportación no reside solo en reunir información, sino en ordenarla según su utilidad dentro del proceso de construcción y evaluación de un equipo.

A ello se añade un rasgo diferencial del proyecto: el uso de recursos propios generados procedualmente. Esta decisión no solo responde a una cuestión de identidad visual o de diferenciación estética, sino también a una necesidad práctica relacionada con la independencia del proyecto respecto a recursos externos y con la voluntad de reducir riesgos derivados del uso de materiales oficiales. Desde el punto de vista del caso de estudio, esta característica mejora soluciones existentes en las que el apartado visual depende de fuentes externas o de recursos no controlados directamente por la aplicación, ya que permite mantener una línea propia y un mayor dominio sobre los contenidos que se muestran.

El segundo gran bloque del caso de estudio es el generador de equipos, que constituye el núcleo funcional de TrainerForge. Mientras que algunas herramientas del mercado se centran exclusivamente en mostrar debilidades y resistencias o en montar un equipo de forma muy básica, aquí se propone una sección en la que cada uno de los seis slots del equipo puede configurarse con un nivel de detalle mucho mayor. Cada Pokémon podrá asociarse a movimientos, habilidades, objetos equipados y otros datos de combate, de manera que el equipo deje de ser una simple lista de nombres y pase a convertirse en una representación útil del planteamiento real del jugador. Este cambio es importante porque acerca la herramienta a la práctica cotidiana del usuario y no solo a una fase preliminar de selección.

Además, el generador de equipos no se orienta únicamente al combate competitivo estándar. El proyecto contempla opciones optimizadas para retos autoimpuestos, como equipos temáticos, monotype o nuzlocks, que forman parte habitual de la experiencia de muchos jugadores pero rara vez reciben un tratamiento específico en herramientas generalistas. Con ello, TrainerForge amplía el marco de uso y da respuesta a formas de jugar que suelen quedar fuera de las aplicaciones centradas en metajuego, estadísticas de torneos o simulación estricta. Esta capacidad de adaptación a distintos estilos de partida constituye una mejora clara frente a soluciones demasiado especializadas, ya que reconoce que no todos los usuarios construyen equipos con los mismos fines.

Otro aspecto relevante del generador es la posibilidad de descargar equipos en distintos formatos. Este detalle, aunque aparentemente secundario, mejora la utilidad real de la herramienta, ya que permite conservar configuraciones, consultarlas fuera de la web o reutilizarlas posteriormente sin depender siempre de una sesión activa. En comparación con otras herramientas que resuelven la planificación solo dentro de su propio entorno, esta funcionalidad añade portabilidad y refuerza la idea de TrainerForge como espacio de trabajo personal y no solo como visor temporal.

Un tercer bloque fundamental es la incorporación de un sistema de perfil de usuario a través de la Tarjeta de Entrenador. Esta pieza resulta especialmente significativa porque introduce una dimensión de persistencia y personalización que muchas herramientas del mismo ámbito no desarrollan con profundidad. La Tarjeta de Entrenador no se concibe solo como una ficha visual, sino como el punto de acceso a la identidad del usuario dentro de la plataforma, permitiendo reflejar datos personales, preferencias y elementos simbólicos inspirados en la estética de la franquicia. Así, el proyecto mejora soluciones puramente funcionales al integrar una capa de apropiación personal que favorece la continuidad de uso y la vinculación del jugador con la herramienta.

Ligado a ello aparece el sistema de guardado de contenido. Una de las principales limitaciones de muchas herramientas de consulta o planificación es que resuelven una tarea inmediata, pero no ayudan a conservar de manera organizada lo que el usuario ha construido. TrainerForge se propone corregir esta carencia permitiendo asociar al perfil equipos guardados, registro de Pokémon capturados, videojuegos en posesión y otros elementos relacionados con el progreso del jugador. Gracias a ello, la aplicación deja de ser un recurso de consulta puntual y pasa a convertirse en un espacio persistente de gestión personal.

Esta dimensión persistente es precisamente una de las claves con las que TrainerForge mejora el panorama descrito en el estado del arte. Mientras que una base de datos ofrece consulta, un team builder aporta composición y un simulador permite ensayo, TrainerForge incorpora continuidad entre unas funciones y otras, conservando la información producida por el usuario y dándole un valor acumulativo dentro del sistema. Desde el punto de vista del diseño de producto, esto supone pasar de herramientas de uso aislado a una plataforma que acompaña procesos más largos y más personales.

El simulador de combate previsto para la versión 1.0 responde a otra necesidad concreta detectada durante el análisis previo. No todos los usuarios necesitan una simulación exhaustiva y competitiva del sistema de combate, como la que puede ofrecer una plataforma especializada, pero sí resulta útil disponer de una valoración orientativa que permita comparar equipos y hacerse una idea general de su rendimiento. Por eso, TrainerForge opta por un simulador representativo, capaz de determinar un ganador entre equipos existentes en modalidades como 1 contra 1, 2 contra 2 o 6 contra 6, tomando en cuenta parámetros fijos como niveles o ventajas de tipo. Esta decisión reduce complejidad técnica sin renunciar a una función valiosa desde el punto de vista del usuario.

La mejora, en este caso, no consiste en ofrecer una simulación más profunda que la de las plataformas líderes, sino en ofrecer una simulación más integrada. El equipo que se consulta, configura y guarda dentro de TrainerForge podrá utilizarse posteriormente en el simulador sin abandonar el ecosistema de la aplicación, lo que evita duplicar trabajo o reconstruir configuraciones en otros entornos. De esta forma, la herramienta refuerza la continuidad entre módulos y convierte la simulación en una prolongación natural del proceso de construcción del equipo.

En un plano posterior, el proyecto incorpora también elementos pensados para enriquecer la experiencia y sostener el uso continuado de la plataforma. El sistema de logros, por ejemplo, añade una capa de progresión simbólica que transforma el perfil del usuario en algo más que un contenedor de datos, mientras que las funciones de amistad, intercambio de información del perfil y simulación de GTS abren la puerta a una dimensión social que no suele aparecer integrada en herramientas de este tipo. Estas funciones no son imprescindibles para la resolución del problema inicial, pero sí mejoran la propuesta global al convertir la aplicación en un entorno con más recorrido, más capacidad de retención y más posibilidades de interacción entre usuarios.

El modo oscuro y las mejoras de rendimiento previstas también forman parte del caso de estudio, aunque puedan parecer cuestiones secundarias frente a las funciones principales. Una aplicación pensada para acompañar sesiones de juego prolongadas necesita una interfaz cómoda, clara y adaptable a distintos dispositivos o condiciones de uso. Del mismo modo, una plataforma que aspira a crecer con nuevas secciones debe cuidar desde el inicio la organización del código, la eficiencia de carga y la mantenibilidad, ya que de lo contrario la acumulación de funciones terminaría volviéndose en contra del propio proyecto.

En ese sentido, la arquitectura elegida también forma parte de la solución planteada. El uso de Spring Boot, React, TypeScript y PostgreSQL no es una elección puramente tecnológica, sino una respuesta a las exigencias del caso de estudio: hace falta una base robusta para modelar relaciones complejas, una interfaz dinámica para conectar distintos módulos y una estructura suficientemente clara como para admitir nuevas ampliaciones sin comprometer la estabilidad del sistema. Esa relación entre problema y arquitectura resulta importante, porque el proyecto no persigue únicamente resolver necesidades presentes, sino construir una base sobre la que seguir incorporando funciones en el futuro.

Otro punto que merece atención es la forma en que TrainerForge organiza la evolución del desarrollo. La aplicación está dividida en versiones con objetivos concretos, de manera que primero se establece una base funcional sólida y, sobre ella, se añaden progresivamente nuevas capacidades. Este planteamiento evita intentar abarcar desde el inicio todas las posibilidades del proyecto y permite priorizar aquellas funciones que responden de forma más directa al problema identificado: consulta útil, construcción detallada de equipos, persistencia de contenido y una primera capa de simulación. Desde el punto de vista del caso de estudio, esta secuenciación constituye una mejora metodológica, porque ordena el desarrollo según valor real para el usuario y no únicamente según ambición funcional.

En conjunto, el caso de estudio demuestra que TrainerForge no pretende competir de manera frontal con cada una de las herramientas analizadas en el estado del arte, sino resolver una carencia que aparece entre ellas. Allí donde unas destacan por la profundidad informativa, otras por el análisis defensivo del equipo y otras por la simulación avanzada, TrainerForge plantea una solución centrada en la continuidad de uso, la organización personal y la integración de funciones complementarias dentro de un mismo espacio. Su aportación no depende de una única característica aislada, sino de la combinación de varias decisiones de diseño y funcionalidad que, juntas, permiten responder de forma más natural a las necesidades del jugador durante la partida.

Por ello, la mejora principal que introduce el proyecto puede formularse de la siguiente manera: frente a un ecosistema de herramientas especializadas pero dispersas, TrainerForge propone una plataforma cohesionada, preparada para crecer y orientada al uso práctico. No se limita a ofrecer información, ni se queda en la composición superficial de equipos, ni delega toda la experiencia en sistemas externos, sino que articula una solución propia en la que consulta, planificación, almacenamiento y evolución del usuario comparten la misma lógica de funcionamiento. Esa coherencia interna es, precisamente, la base sobre la que se sustenta todo el caso de estudio.